

Dry Run

static int shifts;

① Insert AT

→ for (inti = size; i > index; i--) {

a[i] = a[i-1]

shifts++;

}

a[index] = value;

arr[] = {1, 2, 4, 5}

4 = size, i = 2, val = 3

④

a[5] = a[4]

a[4] = a[3]

a[3] = a[2]

shift

a[3] = a[2]

a[2] = a[1]

outloop

a[index] = value

{a[2] = 3}

// copy of (a, length)

queue → offer to push

↳ poll to remove

① delete AT

for (inti = index; i < size - 1; i++) {

a[i] = a[i+1]

shifts++

}

a[size-1] = 0

Ex. [3, 9, 8, 7, 2]

0 1 2 3 4 = size = 5

Remove index = 2

a[2] = a[2+1]

2 < 5-1 = 4

[3, 9, 8, 7, 2]

2 < 4

a[3] = a[4]

[3, 9, 7, 2, 2]

a[size-1] = 0

a[4-1] = 0

a[4] = 0

[3, 9, 7, 2, 0]

Circular queue

normal queue

→ [2 | 3 | 4 | 5 | 6]

↑
First

↑
Rear

↳ If at End = 0

If

[| | | 5 | 6]

2 3 4

poll

↳ to refill we do circular queue

Linear Queue

int[] data = { 3, 7, 8, 9, 5, 4 }

private final = 0;
rear = -1;
capacity = 5

enqueue = 3

↓

rear++

data[++rear] = 3

↓

-1 + 01 = 0 → data[0] = 3

enqueue = 7

↓

data[++rear] = 7

↓

0 + 1

data[1] = 7

dequeue → data[front++] = 0

data[0+1] = front++

Array.toString(data) = "[3, 7, 8, 9, 5]

→ [7, 8, 9, 5]

for displaying.

Circular Queue

queue[] = new int [5];

front = -1

rear = -1

is Full

→ front == 0? if rear == max - 1
= 5 - 1
= 4

0 1 2 3 4
3, 7, 8, 9, 5

↑

front == 0

↑

rear == 4

→ Full → Yes

→ rear + 1 == front

(4) + 1 == 0? → Not Full → should be Full

dequeue → [3, 7, 8, 9, 5]

↑

front

↑

rear

rear + 1 == front?

4 + 1

5 == 2

max - 1 = 4

= 0

Enqueue

① [30]

↑↑
F = 0 R = 0

③ [30, 70, 20, 10, 40]

↑↑↑↑
F = 0 R = 4

② [30, 70]

↑↑
F = 0 R = 1

enqueue

↓
No

↓
Full → F = 0
R = 4

④ dequeue

↓

[30, 70, 20, 10, 40]

↓

First = Removed

Removed = 30

Front++ = 1

[30, 70, 20, 10, 40]

↑

Front =

Removed = 70

[30, 70, 20, 10, 40]

↑

Front = 2

↑

R = 4

Removed = 40

F = -1

→ R = -1

Removed = 20, 10

[30, 70, 20, 10, 40]

↑↑
R = 0

[, , , 10, 40] 50 110 130

↑↑↑↑
F = 3 R = 4 → 0 1 2 3 4
R

enqueue →

F = 3, R = 2

→ R + 1 = F

3 = 3 → Full ✓

3) 1